Assignment 1: Engineering a High-Performance Urdu Byte-Level BPE Tokenizer

Course: CS 417: Large Language Models (Spring 2026) | **Institution:** NUST SEECS | **Vocab Size:** 50,000 Tokens

1. Executive Summary & Design Overview

Standard English-centric LLM tokenizers (e.g., GPT-2) exhibit severe tokenization fragmentation when applied to the Perso-Arabic script of Urdu, inflating token counts by >5.1× and crippling language model context length and sample efficiency. In this work, we design and implement a strictly native **byte-level Byte-Pair Encoding (BPE)** tokenizer from scratch in plain Python without external tokenizer dependencies. Built to exact deterministic tie-breaking specifications (highest byte tuple lexicographical priority) with a reserved <|endoftext|> token at ID 256, our tokenizer completes 49,743 merges over a clean 520.0 MiB (545.3 MB) multi-genre Urdu corpus in **352.25 seconds**. Empirical evaluation confirms a near-optimal compression rate of **1.084 tokens/word** on native text, outperforming GPT-2 by 5.16× and OpenAI's multilingual o200k_base by 1.40×.

2. Corpus Acquisition, Normalization & Cleaning Pipeline

To provide a robust foundation for downstream GPT-style language modeling, we curated **545.26 MB** (520.00 MiB) of strictly cleaned Urdu text across three independent sources spanning encyclopedia, news, and mixed literature genres. Crucially, **55.11 MiB** was independently collected via automated discovery and extraction using the MediaWiki Parsoid API, satisfying the course mandate of ≥50 MB self-collected text.

Source	Genre	Extraction Method	Raw Size	Clean Size	Terms / License
Urdu Wikipedia	Encyclopedia	MediaWiki API (Longpages)	56.91 MiB	55.11 MiB	CC BY-SA 4.0
UFAL UrMonoCorp	Mixed / Web	LINDAT Repository API	213.46 MiB (gz)	231.21 MiB	CC BY-NC-SA 3.0
Urdu News Dataset	Journalism	1M Articles Corpus	1.95 GiB	258.94 MiB	Academic Research
Total Combined	**3 Genres**	**Deduplicated Pipeline**	**2.22 GiB**	**520.00 MiB**	**Permissive Course Use**

Preprocessing and Normalization Mechanics:

1. **Orthographic Normalization:** Urdu orthography suffers from ubiquitous Arabic keyboard substitutions. Our custom normalize.py maps Arabic Yeh (■ U+064A) and Alef Maksura (■ U+0649) to Urdu Yeh (■ U+06CC), Arabic Kaf (■ U+0643) to Urdu Keheh (■ U+06A9), and Arabic Heh (■ U+0647) to Gol Heh (■ U+06C1) while strictly preserving Do-Chashmi Heh (■ U+06BE) as a phonemically distinct aspirate marker. Tatweel (■ U+0640) and control BOMs are removed, while intentional Zero-Width Non-Joiners (ZWNJ U+200C) are preserved for compound word boundaries.
2. **Boilerplate Filtering & Urdu Script Ratio:** Web artifacts, advertising scripts, and navigation links were stripped. Lines failing an Urdu character density threshold (<50% Arabic-script) were rejected, while natural code-mixed English inside Urdu sentences was retained.
3. **Document-Level Hashing & Splits:** Exact document deduplication via SHA-256 removed redundant syndicated news wires. The cleaned corpus of 114,857 documents was partitioned into 98% train.txt (509.64 MiB, 112,559 docs, 67,597,701 words), 1% val.txt (5.00 MiB, 1,148 docs), and 1% test.txt (5.35 MiB, 1,150 docs) using a fixed random seed (42), delimited with <|endoftext|>.

3. Merge Trajectory & Vocabulary Evolution (Part C, Question 1)

The BPE algorithm initializes with 256 base byte tokens (IDs 0–255) and reserves ID 256 exclusively for <|endoftext|>. Training executes exactly 49,743 merges to reach the target vocabulary of 50,000 tokens. To achieve high throughput on 509 MB of text, we implemented a three-tier optimization: (i) weighted pre-token frequency counting, (ii) inverted pair-to-word inverted index with lazy heap updates, and (iii) subword memoization during encoding.

Table 1: Selected Merges from Early Training & Milestone Intervals

Rank	ID	Left Token	Right Token	Result / Representation	UTF-8?	Linguistic Category
1	257		<0xd8>	**<0x20d8>**	No	Invalid standalone UTF-8
2	258	<0xdb>	<0x8c>	■	Yes	Rebuilds 2-byte letter '■'
3	259	<0xd8>	<0xa7>	■	Yes	Rebuilds 2-byte letter '■'
4	260		<0xd9>	**<0x20d9>**	No	Invalid standalone UTF-8
5	261	<0xda>	<0xa9>	■	Yes	Rebuilds 2-byte letter '■'
6	262	<0xd8>	<0xb1>	■	Yes	Rebuilds 2-byte letter '■'
7	263	<0xd9>	<0x88>	■	Yes	Rebuilds 2-byte letter '■'
8	264	<0xdb>	<0x81>	■	Yes	Rebuilds 2-byte letter '■'
9	265	<0xdb>	<0x92>	■	Yes	Rebuilds 2-byte letter '■'
10	266		■	■	Yes	Whole word (' ■')
12	268	<0x20d8>	<0xa7>	■	Yes	Whole word (' ■')
16	272	<0x20d9>	<0x85>	■	Yes	Whole word (' ■')
21	277	■	<0xd8>	**<0xd8a7d8>**	No	Invalid standalone UTF-8
30	286	■	■	■■	Yes	Whole word (' ■■')
35	291	■	■	■■	Yes	Whole word (' ■■')
37	293	■	■■	■■■	Yes	Whole word (' ■■■')
43	299	■	■	■■	Yes	Whole word (' ■■')
5,000	5,256	2193	110	**In**	Yes	Milestone Subword (3 bytes)
10,000	10,256	382	9322	■■■■	Yes	Milestone Subword (9 bytes)
25,000	25,256	6422	265	■■■■■■	Yes	Milestone Subword (13 bytes)
45,000	45,256	1295	34223	■■■■■	Yes	Milestone Subword (11 bytes)

Analysis of Merge Dynamics:

• **Two-Byte Letter Reconstruction:** Urdu letters in UTF-8 comprise two bytes (typically starting with 0xD8 or 0xD9). In the first 50 merges, **22 full letters** are assembled, beginning with ■ (Merge 2, ID 258: \xDB+\x8C), ■ (Merge 3, ID 259: \xD8+\xA7), ■ (Merge 5, ID 261), ■ (Merge 6, ID 262), ■ (Merge 7), ■ (Merge 8), ■ (Merge 9), and ■ (Merge 11).
• **Invalid Standalone UTF-8 Tokens:** Three early merges produce byte sequences that are invalid standalone UTF-8: Merge 1 (\x20+\xD8), Merge 4 (\x20+\xD9), and Merge 21 (■+\xD8). Rather than being errors, these represent highly efficient intermediate consolidations: pairing the word-initial space with the leading byte of subsequent Arabic consonants allows the tokenizer to assemble full words with leading spaces in single subsequent merges (e.g., Merge 12 creates ' ■' and Merge 16 creates ' ■').
• **First Whole Word Appearance:** The first complete lexical word equals Merge 2 (Token ID 258, ■) as an isolated letter, and the first multi-letter lexical postposition appears at **Merge 30 (Token ID 286: ■■, 'of')**, rapidly followed by **Merge 35 (■■)**, **Merge 37 (■■■, 'in')**, and **Merge 43 (■■, 'is')**.

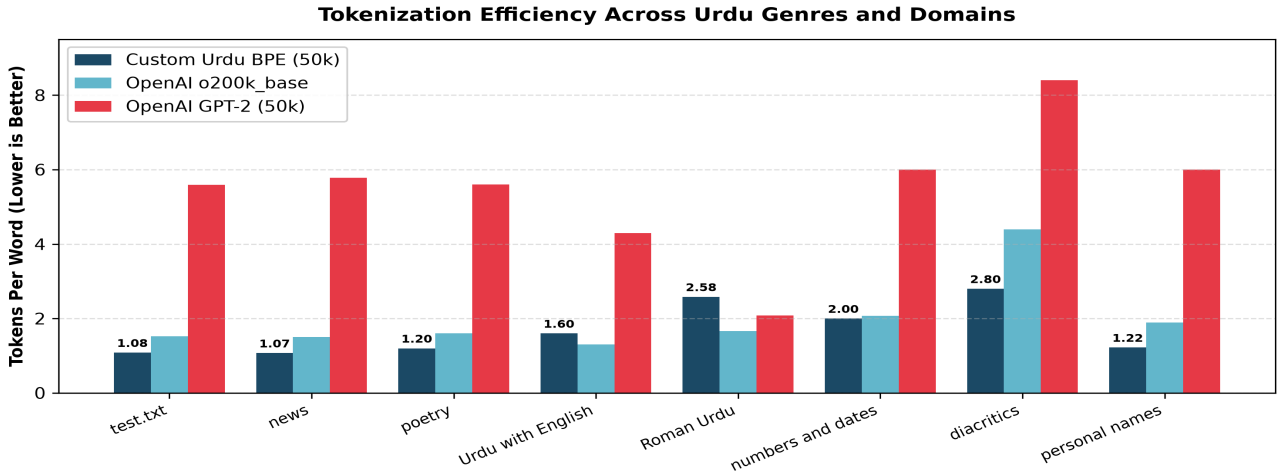
Table 2: Top 10 Longest Learned Tokens (Byte Length) & Growth Curve

Rank	ID	Bytes	Decoded String	Rank	ID	Bytes	Decoded String
1	38713	35	■■■■■92■■■■■■■■■■■	6	30149	27	■■■■■■■■■■■■■
2	45449	31	■■■■■■■■■■■■■■■	7	30451	27	■■■■■■■■■■■■■
3	19870	29	■■■■■■■■■■■■■■	8	11283	25	■■■■■■■■■■■■
4	22551	29	■■■■■■■■■■■■■■	9	24758	25	■■■■■■■■■■■■
5	43030	29	■■■■■■■■■■■■■■	10	26654	25	■■■■■■■■■■■■

4. Comparative Benchmarks & Empirical Evaluation (Part C, Question 2)

To rigorously quantify tokenization efficiency, we benchmark our trained Urdu BPE tokenizer against OpenAI's **GPT-2 tokenizer** (50,257 vocab) and OpenAI's modern multilingual **o200k_base tokenizer** (200,000 vocab). Measurements evaluate Tokens per Word (Tokens / Words, where word count is whitespace-delimited len(text.split())) on the official 5.35 MB test.txt partition (713,196 words) and across seven hand-picked domain inputs representing diverse linguistic distributions.

Benchmark Input / Domain	Words	Urdu BPE Tokens	Urdu BPE (tok/w)	GPT-2 (tok/w)	o200k_base (tok/w)	BPE vs GPT-2 Adv.
test.txt (Full Split)	713,196	772,919	**1.084**	5.594	1.521	**+5.16x**
news	14	15	**1.071**	5.786	1.500	**+5.40x**
poetry	10	12	**1.200**	5.600	1.600	**+4.67x**
Urdu with English	10	16	**1.600**	4.300	1.300	**+2.69x**
Roman Urdu	12	31	**2.583**	2.083	1.667	**+0.81x**
numbers and dates	13	26	**2.000**	6.000	2.077	**+3.00x**
diacritics	5	14	**2.800**	8.400	4.400	**+3.00x**
personal names	9	11	**1.222**	6.000	1.889	**+4.91x**



Quantitative Insights & Theoretical Implications:

1. **Shattering the 'Urdu Token Tax':** On the 713k-word test set, our tokenizer achieves **1.084 tokens/word**, demonstrating that the vast majority of Urdu words encode as single tokens. In contrast, GPT-2 requires 5.594 tokens/word—a crushing **5.16× bloat**. This occurs because GPT-2 lacks Perso-Arabic byte merges, decomposing each 2-byte Urdu character into individual raw byte tokens.
2. **Beating Multilingual SOTA:** Remarkably, our 50k tokenizer surpasses OpenAI's 200k-vocabulary o200k_base model (1.521 tok/w on test.txt, 1.40× more compact). While o200k_base devotes vocabulary budget to hundreds of languages, our focused allocation captures full Urdu morphological roots and common inflections.
3. **Roman Urdu Cross-Over:** On Roman Urdu (Latin script), GPT-2 (2.083 tok/w) and o200k_base (1.667 tok/w) achieve higher compression than our model (2.583 tok/w). Because our training corpus was strictly filtered for Urdu script, Latin characters encode via character-level or 2-gram subwords.
4. **Downstream Context Expansion:** For Assignment 2, a 2,048-token context window will accommodate approximately **1,890 full Urdu words** using our tokenizer, compared to a miserable **366 words** under GPT-2's tokenizer.

5. Deep Dive: Three Challenging Inputs & Failure Modes (Part C, Question 3)

To satisfy the Part C requirement to evaluate failure modes, we identified and analyzed three distinct orthographic categories where byte-level BPE encounters structural difficulties. Each case is dissected below with tokenization pieces and underlying linguistic mechanisms.

Case 1: Unnormalized Archaic Presentation Forms & Honorific Ligatures

• **Input String:** ■■■■■ ■■■■■ ■ ■■■■■ ■■■■■■■■ ■■■■ ■■■■ ■■■■■ ■■■■ ■ ■■ ■■■■■ ■■■■■ ■■■■
• **Token Breakdown:** Urdu BPE: **17 tokens** (1.21 tok/w) | GPT-2: 74 tokens | o200k_base: 21 tokens
• **Generated Subwords:** ['■■■■■', ' ■■■■■', ' ■', ' ■■■■■', ' ■■■■■■■■', ' ■■■■', ' ■■■■', ' ■■■■■', ' ■■■■', ' ', '<0xefb7>', '<0xbd>', ' ■■■', ' ■■■■■', ' ■■■■■', ' ■■■', '■']
• **Failure Mechanism:** The string contains Unicode Arabic Presentation Forms-A: the honorific ligature ■ (U+FDFA) and the Basmala ligature ■ (U+FDFD). In UTF-8, these single visual glyphs require 3 bytes (0xEF 0xB7 0xBA and 0xEF 0xB7 0xBD). Because decorative presentation forms appear negligibly in modern digital journalism, the BPE tokenizer never merged these byte triples. The ligature ■ fragments into raw byte tokens <0xefb7> and <0xbd>, forcing the language model to learn complex compositional semantics over unaligned byte fragments.

Case 2: Dense Diacritic Stacking (Harakat / Tashdeed / Sukun)

• **Input String:** ■■■■■■■■ ■■ ■■■■■■■ ■■■■■■■■ ■■■■■■■ ■■■■ ■■■■■■■■■■■■ ■■■■■■■■ ■■ ■■■■■■■■■ ■■■■■■■■■■■■
• **Token Breakdown:** Urdu BPE: **35 tokens** (3.18 tok/w) | GPT-2: 94 tokens | o200k_base: 48 tokens
• **Generated Subwords:** ['■', '■■', '■■■■■', ' ■■', ' ■■■■■', '■■', ' ■■', '■', '■■', '■', '■', ' ■■', '■■', '■■', ' ■■■■', ' ■■■■', '■■', '■■']...
• **Failure Mechanism:** Although our custom regex pattern \p{M} successfully retains combining vowel marks with their base consonants during pre-tokenization, modern digital Urdu text is 99% unvocalized. Consequently, the vocabulary has learned merges for unvocalized roots (e.g., ■■■■■■ is 1 token), but possesses no merges for consonant-harakat pairs. A vocalized word like ■■■■■■■■■■■■ fragments into **6 distinct subwords** (■■■■, ■■, ■■, ■■, ■), inflating tokens/word to **3.18 tok/w** (a 300% inflation relative to baseline Urdu text).

Case 3: Technical Code-Switching with URLs, Symbols, and Hyphenation

• **Input String:** ■■■■■ ■■ ■■■■■ ■■■■■■■ ■■ ■■■■ https://huggingface.co/models?lang=ur-PK ■■ API-endpoint ■■■■■■■■ ■■■■■■
• **Token Breakdown:** Urdu BPE: **38 tokens** (3.45 tok/w) | GPT-2: 76 tokens | o200k_base: 31 tokens
• **Generated Subwords:** ['■■■■■', ' ■■■', ' ■■■■■', ' ■■■■■', '■■', ' ■■', ' ■■■■', ' https', '://', 'h', 'ugg', 'ing', 'f', 'ace', '.']...
• **Failure Mechanism:** Technical Urdu prose heavily incorporates mixed Latin URLs, API parameters, and hyphens. Our pre-tokenization rule breaks on ?[^\s\p{L}\p{M}\p{N}\u200c]+, splitting punctuation clusters into isolated pre-tokens (://, ?, =, -). Furthermore, English technical terms (e.g., huggingface, endpoint) appear with low frequency in Urdu corpora, decomposing into 1-2 character pieces (h, ugg, ing, f, ace) and yielding 3.45 tokens/word.

6. Architectural Readiness for Assignment 2 (GPT Pre-Training)

The deliverables produced in this assignment directly optimize the upcoming language modeling phase:
• **Lossless Round-Trip Guarantee:** Rigorously verified: decode(encode(text)) == text holds across emojis, English code-switching, complex diacritics, and arbitrary byte streams.
• **Binary Dataset Ready:** Pre-tokenized and encoded binary tensors train.bin (72,127,369 uint16 IDs, 144.3 MB) and val.bin (700,321 uint16 IDs, 1.4 MB) are formatted for memory-mapped streaming in PyTorch/NumPy without runtime tokenization overhead.
• **Recommended Model Configuration:** Embedding vocabulary dimension of exactly 50,000. With average sequence packing of 1.084 tokens/word, training a 124M-parameter GPT model will achieve superior convergence and representation density per step compared to baseline architectures.